# Projet STATISTIQUE AVEC R - Modélisation et analyse des séries temporelles des locations de vélos en libre-service en France

Yester MKRTCHYAN

## Table des matières

**1 Introduction** **2**
1.1 Librairies . . . . . 3

**2 Question 1 : Restructuration Des Données** **3**
2.1 Importation des données . . . . . 3
2.2 Mise en forme et fusion . . . . . 3
2.3 Vérification et visualisation . . . . . 4

**3 Question 2 : Inférence Et Prévision Pour Paris** **6**
3.1 Analyse Exploratoire . . . . . 6
3.2 Modélisation SARIMA . . . . . 11
3.3 Prévision pour 2026 . . . . . 14

**4 Question 3 : Comparaison De La Saisonnalité** **17**

# 1 Introduction

Ce projet étudie les locations de vélos en libre-service dans quatre métropoles françaises : Paris, Lille, Lyon et Marseille. Les données couvrent la période du 1er janvier 2024 au 31 décembre 2025. L'objectif est double. Premièrement, nous souhaitons modéliser la série temporelle des locations à Paris. Nous produisons des prévisions pour les sept premiers jours de l'année 2026. Ces prévisions sont accompagnées d'un intervalle à 90% qui indique la marge d'incertitude. Deuxièmement, nous comparons la composante saisonnière des quatre villes. Nous cherchons à savoir si leur fréquentation hebdomadaire évolue de manière identique ou non. Le rapport s'articule en trois parties. La première partie présente la restructuration des données. La deuxième partie détaille la modélisation et les prévisions pour Paris. La troisième partie compare les profils saisonniers des quatre villes.

### 1.1 Librairies

On charge les librairies nécessaires pour la manipulation des données, la modélisation et les graphiques.

```
library(tidyverse)
library(lubridate)
library(forecast)
library(tseries)
library(knitr)
```

## 2 Question 1 : Restructuration Des Données

### 2.1 Importation des données

On commence par importer les deux fichiers. Le fichier ‘velos_nord.csv‘ contient les données de Paris et Lille. Le fichier ‘velos_sud.csv‘ contient celles de Lyon et Marseille. Dans le fichier nord, les dates sont au format ‘jour_mois_année‘ (par exemple ‘01_Jan_2024‘). On les convertit au format date standard avec la fonction ‘dmy()‘. En regardant les données, on détecte une valeur anormalement négative pour Paris (-791698). Il s’agit probablement d’une erreur de saisie. On remplace cette valeur par la médiane des autres valeurs de Paris. Cela permet de corriger l’erreur sans supprimer la ligne ni modifier trop fortement la série.

```
# Import Nord
nord <- read_csv("velos_nord.csv") %>%
  mutate(Date = dmy(str_replace_all(Date, "_", "/")))

# Traitement de la valeur aberrante de Paris
mediane_paris <- median(nord$Paris[nord$Paris > 0], na.rm = TRUE)
nord <- nord %>%
  mutate(Paris = ifelse(Paris < 0, mediane_paris, Paris))

# Import Sud
sud <- read_csv("velos_sud.csv")
```

### 2.2 Mise en forme et fusion

Les données sont actuellement au format “large” : chaque ville a sa propre colonne. Pour obtenir un format Tidy, on transforme les données au format “long” avec la fonction `pivot_longer()`. On obtient ainsi une colonne `ville` et une colonne `valeur`, ce qui est beaucoup plus pratique

pour l'analyse. On fusionne ensuite les deux tables (nord et sud) avec `bind_rows()`. Enfin, on ajoute une colonne `jour` qui indique le jour de la semaine correspondant à chaque date. Cela permettra d'étudier plus facilement le comportement hebdomadaire des locations.

```
# Format Tidy
nord_tidy <- nord %>%
  pivot_longer(cols = -Date, names_to = "ville", values_to = "valeur")

sud_tidy <- sud %>%
  pivot_longer(cols = -Date, names_to = "ville", values_to = "valeur")

# Fusion
data <- bind_rows(nord_tidy, sud_tidy) %>%
  mutate(
    valeur = as.numeric(valeur),
    jour = wday(Date, label = TRUE, abbr = FALSE, week_start = 1)
  ) %>%
  arrange(ville, Date)
```

## 2.3 Vérification et visualisation

On vérifie la structure des données avec `glimpse()`. On calcule les minima et maxima par ville et on confirme que Paris n'a plus de valeur négative car la valeur minimale est 2794. Enfin, on trace un graphique des séries temporelles pour les quatre villes.

```
# Vérification


glimpse(data)
```

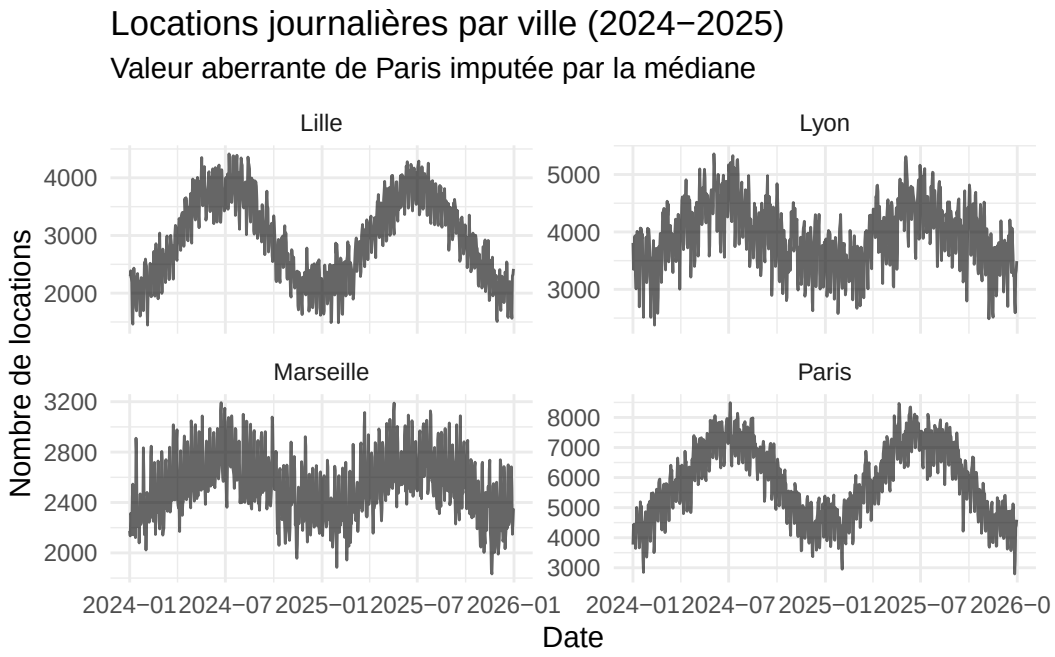
```
Rows: 2,924
Columns: 4
$ Date   <date> 2024-01-01, 2024-01-02, 2024-01-03, 2024-01-04, 2024-01-
05, 20~
$ ville  <chr> "Lille", "Lille", "Lille", "Lille", "Lille", "Lille", "Lille", ~
$ valeur <dbl> 2291, 2292, 2388, 2269, 2189, 1536, 1462, 2161, 2434, 2222, 221~
$ jour   <ord> lundi, mardi, mercredi, jeudi, vendredi, samedi, dimanche, lund~
```

```
#pour vérifier que Paris n'a plus de valeurs négatives
data %>%
  group_by(ville) %>%
  summarise(min = min(valeur), max = max(valeur))
```

```
# A tibble: 4 x 3
  ville       min   max
  <chr>     <dbl> <dbl>
1 Lille      1446  4415
2 Lyon       2378  5359
3 Marseille  1833  3192
4 Paris      2794  8490
```

```r
# Graphique


data %>%
  ggplot(aes(x = Date, y = valeur)) +
  geom_line(alpha = 0.6) +
  facet_wrap(~ville, scales = "free_y") +
  theme_minimal() +
  labs(title = "Locations journalières par ville (2024-2025)",
       subtitle = "Valeur aberrante de Paris imputée par la médiane",
       y = "Nombre de locations",
       x = "Date")
```



Il y a une saisonnalité annuelle marquée, avec une augmentation progressive des locations jusqu'à l'été, suivie d'une diminution en hiver, phénomène cohérent avec un usage plus fréquent des vélos lorsque les conditions météorologiques sont favorables. Des différences apparaissent toutefois entre les villes : Paris se distingue par des volumes plus élevés et une forte amplitude des variations, Lyon présente une variabilité importante, Lille affiche une saisonnalité régulière, tandis que Marseille semble plus stable. À une échelle plus fine, des fluctuations rapides suggèrent également une saisonnalité hebdomadaire, caractérisée par une activité plus soutenue en semaine et plus faible le week-end. Enfin, aucune tendance de long terme nette n'est observable sur la période, la dynamique étant principalement guidée par ces composantes saisonnières, ce qui justifie l'utilisation d'un modèle de type SARIMA intégrant une saisonnalité hebdomadaire.

# 3 Question 2 : Inférence Et Prévision Pour Paris

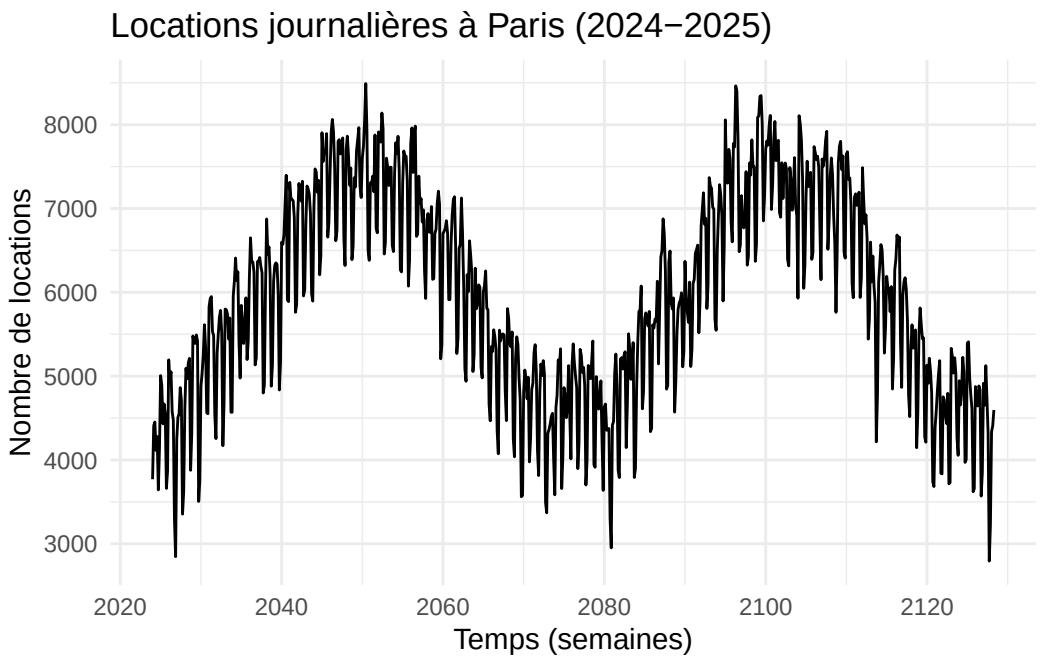
## 3.1 Analyse Exploratoire

On extrait d'abord les données de Paris. On les convertit en objet série temporelle avec une fréquence de 7 jours, ce qui correspond à la saisonnalité hebdomadaire. On trace ensuite le graphique de la série brute pour visualiser son évolution.

```
#  Extraction de la série Paris
paris_ts <- data %>%
  filter(ville == "Paris") %>%
  arrange(Date) %>%
  pull(valeur)

paris_ts_obj <- ts(paris_ts,
                   start = c(2024, 1),
                   frequency = 7)

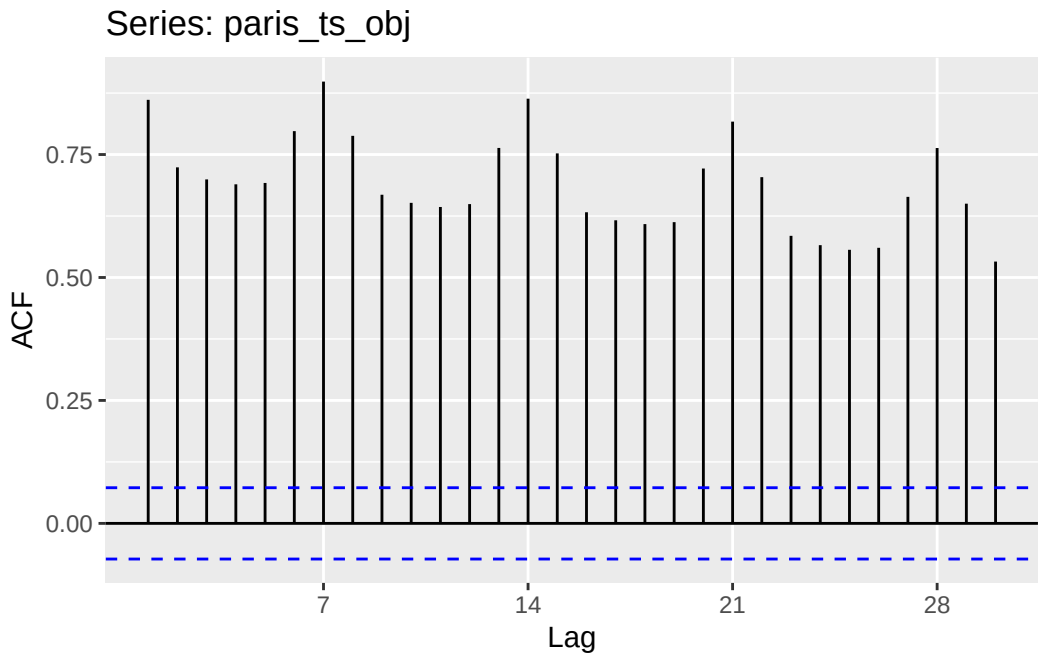
#  Graphique de la série brute
autoplot(paris_ts_obj) +
  labs(title = "Locations journalières à Paris (2024-2025)",
       x = "Temps (semaines)",
       y = "Nombre de locations") +
  theme_minimal()
```



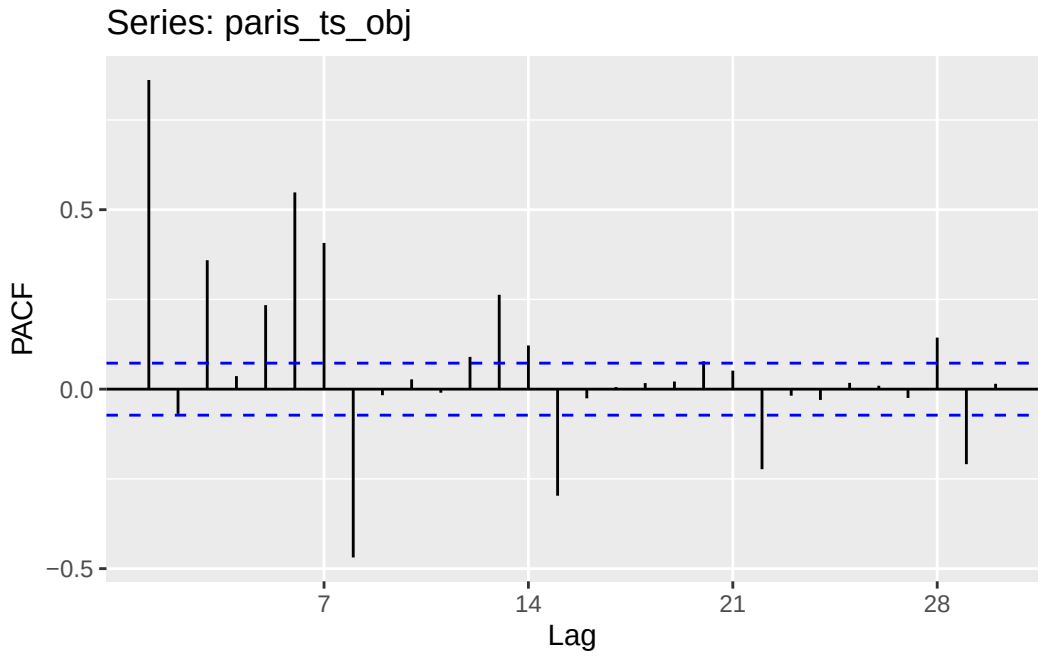

On observe une structure cyclique régulière : les niveaux de fréquentation alternent entre des pics et des creux rapprochés. Cette oscillation correspond à une saisonnalité hebdomadaire. On note également une légère tendance à la hausse sur la période. Ces premières observations justifient le recours à un modèle SARIMA avec une différenciation classique et une différenciation saisonnière.

Ensuite, on regarde l'ACF et la PACF de la série brute. L'ACF diminue lentement, ce qui indique que la série n'est pas stationnaire. La PACF présente un pic important au premier retard, puis devient proche de zéro. On en conclut qu'il faut différencier la série.

```
#  ACF et PACF de la série brute
ggAcf(paris_ts_obj, lag.max = 30)
```



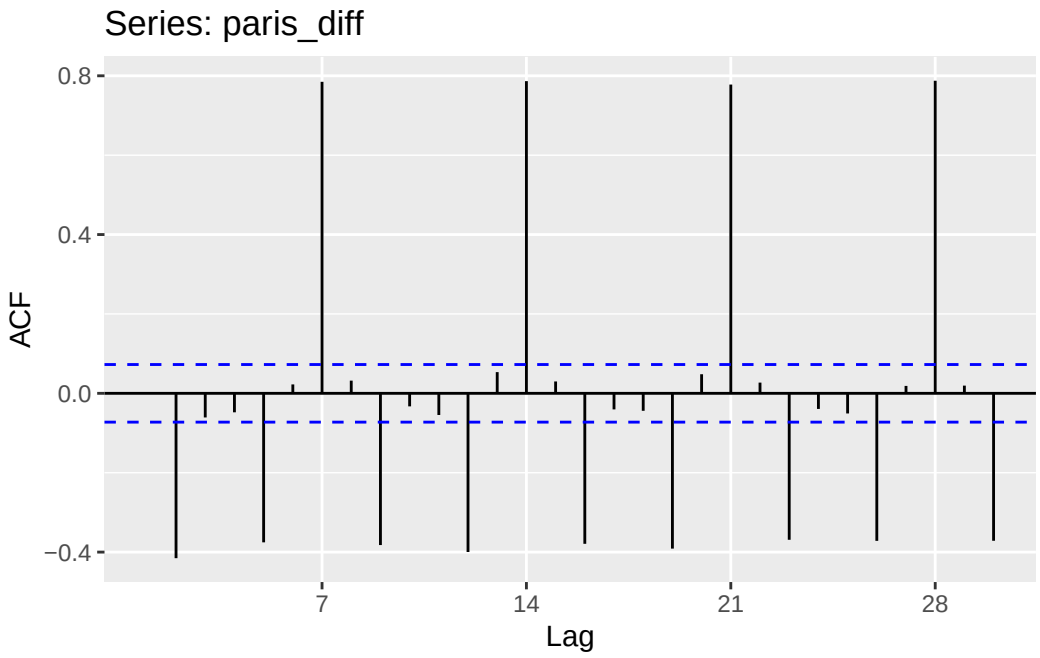

```
ggPacf(paris_ts_obj, lag.max = 30)
```



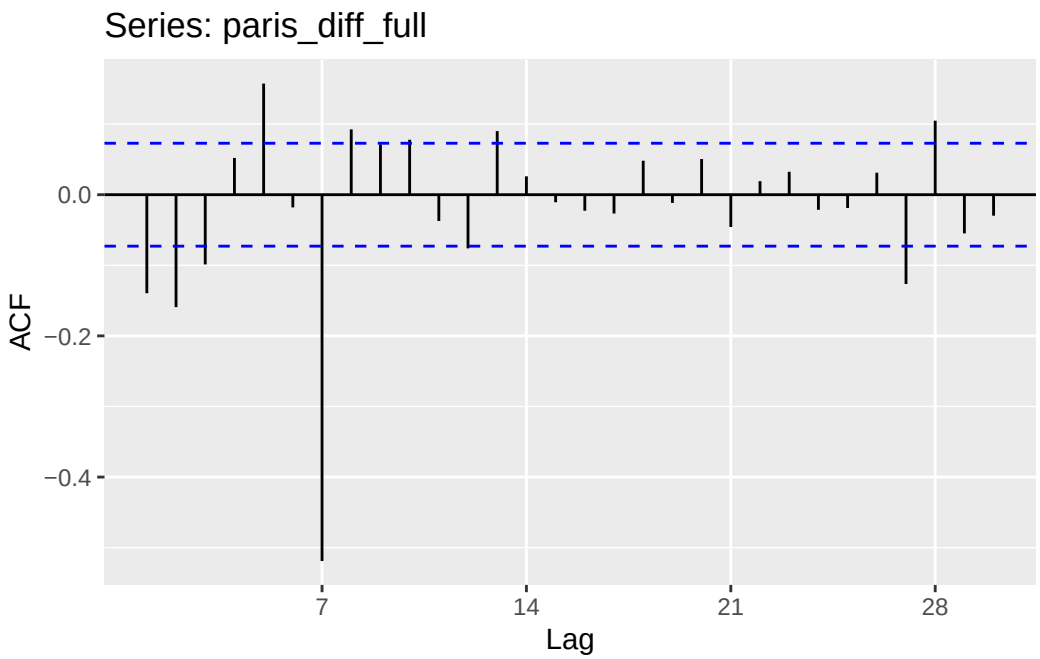

La série brute n'étant pas stationnaire, on applique une première différenciation (d=1). Après l'ACF de la série différenciée présente encore des pics significatifs aux lags 7, 14 et 21. Cela

indique qu'une différenciation saisonnière est également nécessaire. On applique donc une différenciation saisonnière supplémentaire (D=1, s=7) à lag 7.

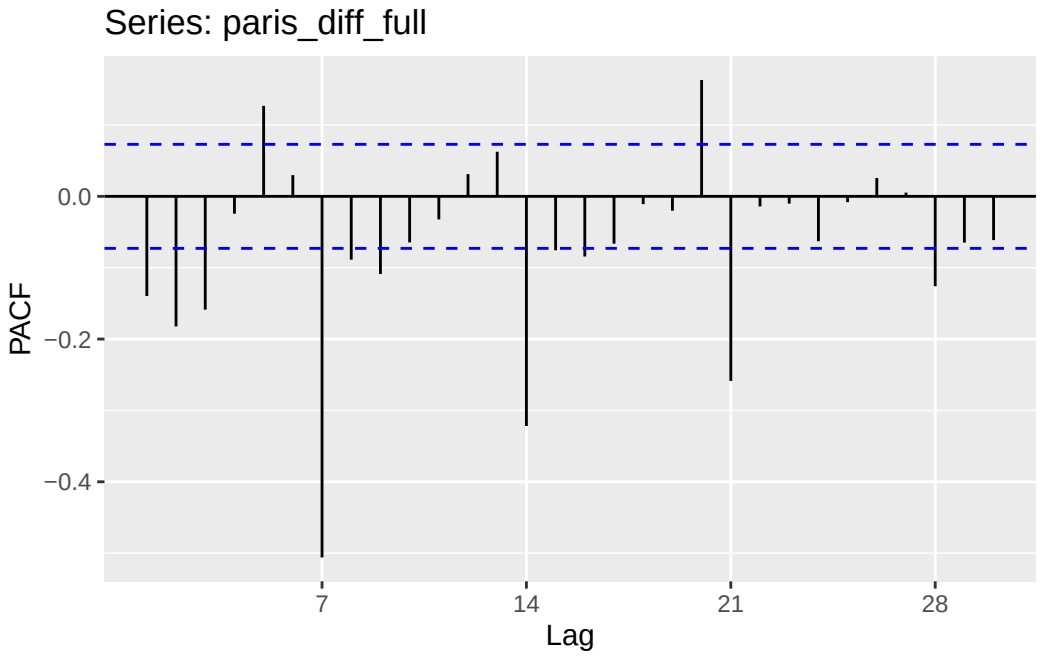
```
#  Série différenciée (d=1)
paris_diff <- diff(paris_ts_obj, differences = 1)
ggAcf(paris_diff, lag.max = 30)
```




```
# Série après double différenciation (d=1 et D=1)
paris_diff_full <- diff(diff(paris_ts_obj, 1), 7)
ggAcf(paris_diff_full, lag.max = 30)
```




```
ggPacf(paris_diff_full, lag.max = 30)
```




Après double différenciation, la série est stationnaire. L'ACF présente un pic net au lag 7, justifiant une moyenne mobile saisonnière d'ordre 1 (Q=1), tandis que le PACF révèle des pics

modérés aux premiers retards non saisonniers et une activité aux retards 7 et 14, orientant la sélection vers un modèle de base de forme SARIMA(p,1,q)(0,1,1)(7).

On choisit alors par exemple SARIMA (1,1,1)(0,1,1)(7) pour tester par la suite.

## 3.2 Modélisation SARIMA

Deux modèles sont mis en concurrence. Le premier, SARIMA(2,0,1)(2,1,0)[7], est celui que propose auto.arima. Le second, SARIMA(1,1,1)(0,1,1)[7], est celui que l'analyse graphique des ACF et PACF a conduit à retenir.

Pour les départager, on utilise deux indicateurs. L'AIC, qui pénalise la complexité : un modèle est d'autant meilleur que son AIC est faible. Le test de Ljung-Box, qui vérifie si les résidus sont non corrélés : on souhaite une p-value supérieure à 0,05 pour conclure à un bruit blanc.

```
# Modèles candidats
modele_auto <- auto.arima(paris_ts_obj, seasonal = TRUE, stepwise = FALSE)
cat("Modèle retenu par auto.arima :\n")
```

```
Modèle retenu par auto.arima :
```

```
print(as.character(modele_auto))
```

```
[1] "ARIMA(2,0,1)(2,1,0)[7]"
```

```
model_recommande <- Arima(paris_ts_obj,
                          order = c(1,1,1),
                          seasonal = list(order = c(0,1,1), period = 7))

# Comparaison des modèles
AIC(modele_auto)          # ← corrigé : modele_auto (avec e)
```

```
[1] 10359.13
```

```
AIC(model_recommande)
```

```
[1] 10192.98
```

```r
Box.test(residuals(modele_auto), lag = 14, type = "Ljung-Box")      # ← corrigé
```

```
    Box-Ljung test

data:  residuals(modele_auto)
X-squared = 43.669, df = 14, p-value = 6.692e-05
```

```r
Box.test(residuals(model_recommande), lag = 14, type = "Ljung-Box")
```

```
    Box-Ljung test

data:  residuals(model_recommande)
X-squared = 18.275, df = 14, p-value = 0.1946
```

```r
# Modèle final (à choisir selon les p-values)
model_final <- model_recommande
summary(model_final)
```

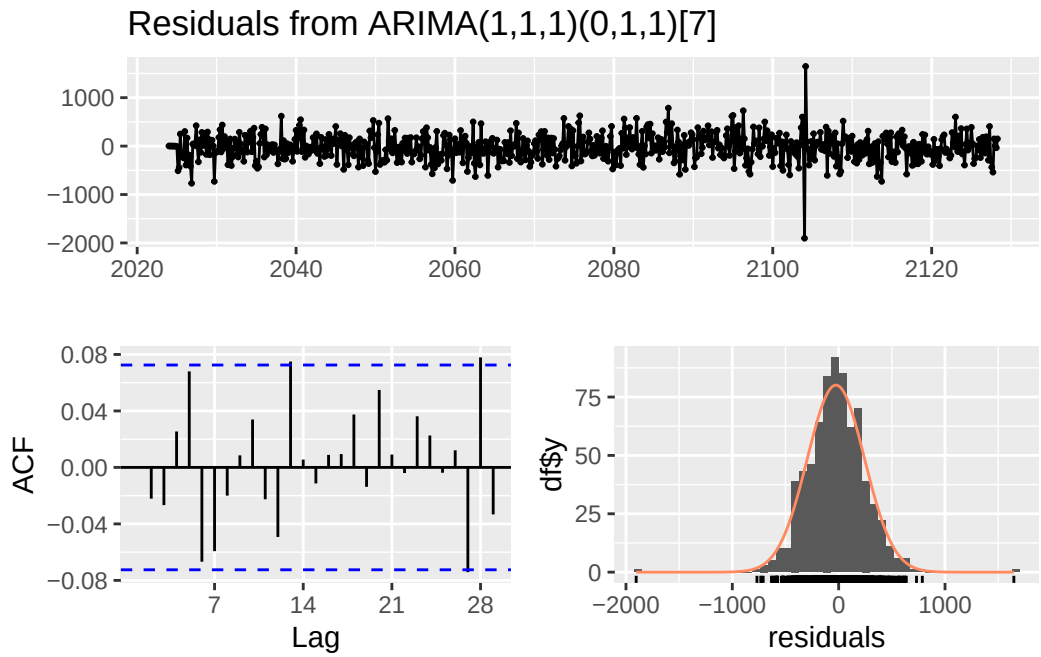
```
Series: paris_ts_obj
ARIMA(1,1,1)(0,1,1)[7]

Coefficients:
         ar1      ma1     sma1
      0.5303  -0.8067  -0.9909
s.e.  0.0600   0.0417   0.0303

sigma^2 = 74245:  log likelihood = -5092.49
AIC=10192.98   AICc=10193.04   BIC=10211.32

Training set error measures:
                    ME    RMSE      MAE        MPE     MAPE      MASE
Training set -25.91876 270.422 204.7743 -0.6345587 3.618318 0.5214318
                     ACF1
Training set 9.713979e-05
```

```r
# Validation des résidus
checkresiduals(model_final)
```




```
    Ljung-Box test

data:  Residuals from ARIMA(1,1,1)(0,1,1)[7]
Q* = 18.275, df = 11, p-value = 0.07542

Model df: 3.   Total lags used: 14
```

```r
Box.test(residuals(model_final), lag = 14, type = "Ljung-Box")
```

```
    Box-Ljung test

data:  residuals(model_final)
X-squared = 18.275, df = 14, p-value = 0.1946
```

C'est donc le modèle SARIMA(1,1,1)(0,1,1)[7] qui est retenu. Son AIC est plus bas et ses résidus ne sont pas corrélés car ils sont dans la bande de non significativité. Puis le test de Ljung-Box indique une p-value de 0,1946 , ce qui signifie que les résidus sont bien du bruit blanc.

### 3.3 Prévision pour 2026

Pour obtenir des intervalles de prévision à 90%, on utilise une méthode par simulation bootstrap. On simule 1000 trajectoires futures sur un horizon de 7 jours. Pour chaque jour, on calcule la médiane des simulations qui constitue la prévision ponctuelle, ainsi que les quantiles à 5% et 95% qui définissent l'intervalle de prévision à 90%.

```
#  Simulations pour les prévisions
h <- 7
n_sim <- 1000
set.seed(123)

simulations <- matrix(NA, nrow = n_sim, ncol = h)
for(i in 1:n_sim) {
  sim <- simulate(model_final, nsim = h, future = TRUE, bootstrap = TRUE)
  simulations[i, ] <- sim
}

borne_inf <- apply(simulations, 2, quantile, probs = 0.05)
borne_sup <- apply(simulations, 2, quantile, probs = 0.95)
prevision_mediane <- apply(simulations, 2, median)
```

Le tableau ci-dessous présente les prévisions pour les sept premiers jours de 2026, avec la borne inférieure et la borne supérieure de l'intervalle de confiance.

```
dates_2026 <- as.Date(c("2026-01-01", "2026-01-02", "2026-01-03",
                        "2026-01-04", "2026-01-05", "2026-01-06", "2026-01-07"))
previsions <- data.frame(
  Date = dates_2026,
  Jour = weekdays(dates_2026),
  Prevision = round(prevision_mediane, 0),
  Inf_90 = round(borne_inf, 0),
  Sup_90 = round(borne_sup, 0)
)

kable(previsions, caption = "Prévisions pour les 7 premiers jours de 2026")
```

Table 1 : Prévisions pour les 7 premiers jours de 2026

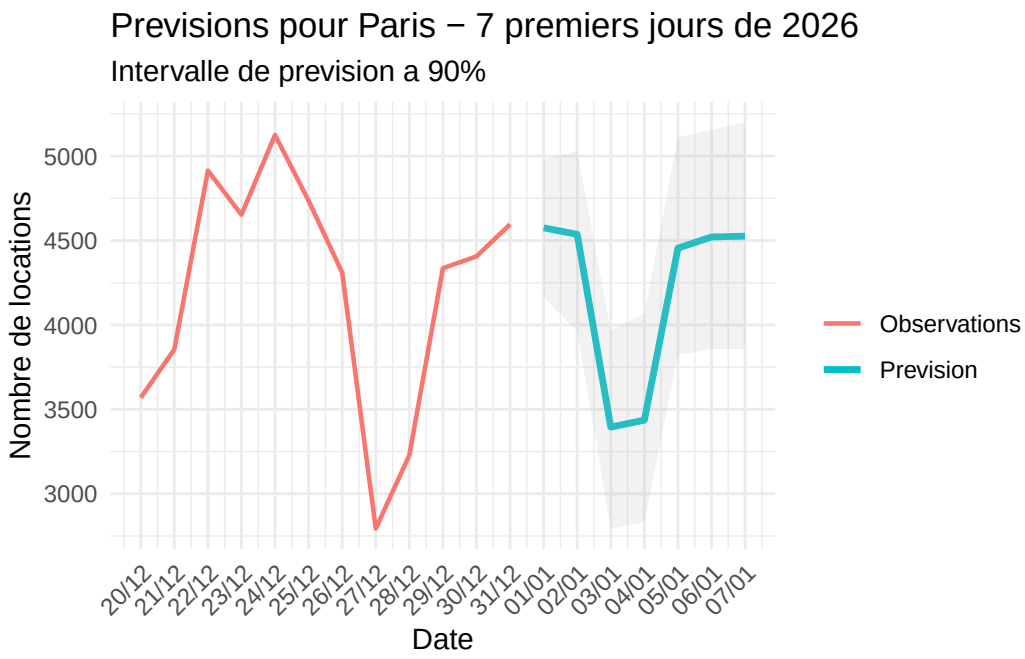
| Date | Jour | Prevision | Inf_90 | Sup_90 |
|---|---|---|---|---|
| 2026-01-01 | jeudi | 4575 | 4164 | 4977 |

| Date | Jour | Prevision | Inf_90 | Sup_90 |
|---|---|---|---|---|
| 2026-01-02 | vendredi | 4537 | 3964 | 5031 |
| 2026-01-03 | samedi | 3395 | 2793 | 3966 |
| 2026-01-04 | dimanche | 3436 | 2834 | 4067 |
| 2026-01-05 | lundi | 4455 | 3821 | 5112 |
| 2026-01-06 | mardi | 4521 | 3858 | 5156 |
| 2026-01-07 | mercredi | 4526 | 3857 | 5201 |

Le graphique met en regard les dernières observations réelles de décembre 2025 avec les prévisions pour les premiers jours de janvier 2026. La courbe de prévision correspond à la médiane des simulations, tandis que la zone grisée représente l'intervalle de prévision à 90 %.

```
#  Graphique final
data_2025 <- data %>%
  filter(ville == "Paris", Date >= "2025-12-20")

ggplot() +
  geom_line(data = data_2025,
            aes(x = Date, y = valeur, color = "Observations"),
            linewidth = 0.8) +
  geom_line(data = previsions,
            aes(x = Date, y = Prevision, color = "Prevision"),
            linewidth = 1.2) +
  geom_ribbon(data = previsions,
              aes(x = Date, ymin = Inf_90, ymax = Sup_90),
              alpha = 0.2, fill = "grey") +
  labs(title = "Previsions pour Paris - 7 premiers jours de 2026",
       subtitle = "Intervalle de prevision a 90%",
       x = "Date", y = "Nombre de locations",
       color = "") +
  scale_x_date(date_breaks = "1 day", date_labels = "%d/%m") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```




On observe un niveau relativement élevé de locations en semaine (1er et 2 janvier), suivi d'une baisse marquée durant le week-end (de 4 537 à 3 395). Par la suite, le nombre de locations augmente de nouveau en début de semaine suivante.

Cette dynamique met en évidence une saisonnalité hebdomadaire, avec une activité plus importante en semaine et plus faible le week-end, ce qui est cohérent avec les observations précédentes. L'élargissement de l'intervalle de prévision traduit une incertitude croissante à mesure que l'horizon de prévision augmente.

## 4 Question 3 : Comparaison De La Saisonnalité

Pour comparer la composante saisonnière des quatre villes, on extrait l'effet hebdomadaire à l'aide d'une décomposition classique additive (fonction `decompose`). Cette méthode sépare la série en trois composantes : tendance, saisonnalité et résidu. On extrait ensuite les sept premiers effets saisonniers correspondant aux jours de la semaine.

```
# Préparer les données
villes <- c("Paris", "Lille", "Lyon", "Marseille")
resultats <- list()

for(i in 1:4) {
  v <- villes[i]

  # Extraire et convertir en ts
  serie_ts <- data %>%
    filter(ville == v) %>%
    arrange(Date) %>%
    pull(valeur) %>%
    ts(frequency = 7)

  # Décomposer
  decomp <- decompose(serie_ts, type = "additive")

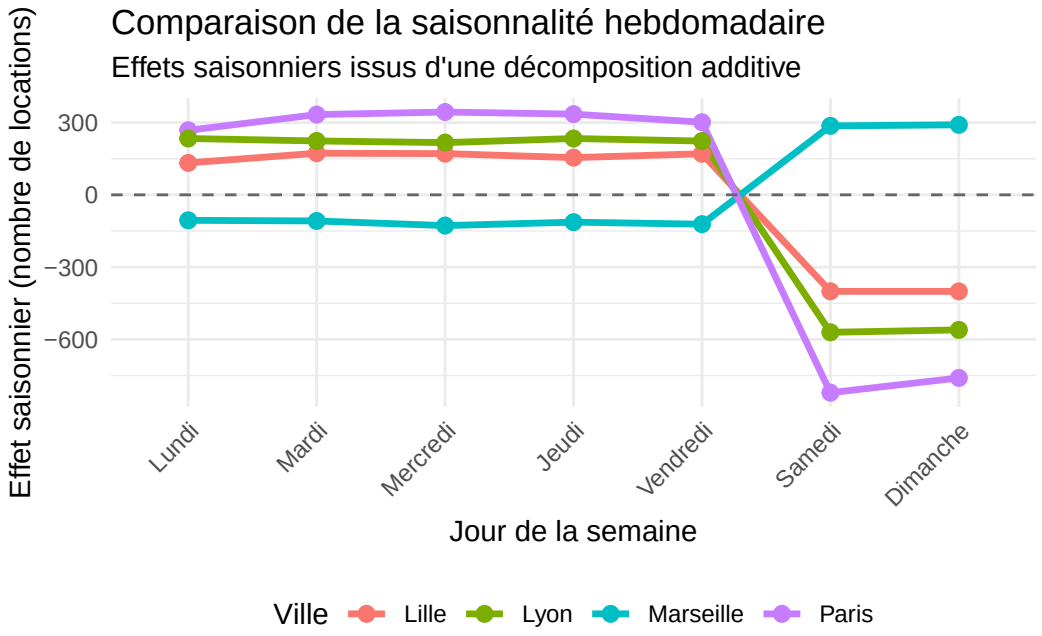
  # Stocker les effets saisonniers
  resultats[[i]] <- data.frame(
    jour = 1:7,
    saison = as.vector(decomp$seasonal[1:7]),
    ville = v
  )
}

# Fusionner
df_saison <- do.call(rbind, resultats)

# Ajouter les noms des jours
jours_fr <- c("Lundi", "Mardi", "Mercredi", "Jeudi", "Vendredi", "Samedi", "Dimanche")
df_saison$jour_label <- factor(df_saison$jour, labels = jours_fr)

# Graphique
ggplot(df_saison, aes(x = jour_label, y = saison, color = ville, group = ville)) +
  geom_line(linewidth = 1.2) +
```

```
  geom_point(size = 2.5) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "gray40") +
  labs(
    title = "Comparaison de la saisonnalité hebdomadaire",
    subtitle = "Effets saisonniers issus d'une décomposition additive",
    x = "Jour de la semaine",
    y = "Effet saisonnier (nombre de locations)",
    color = "Ville"
  ) +
  theme_minimal() +
  theme(
    axis.text.x = element_text(angle = 45, hjust = 1),
    legend.position = "bottom"
  )
```




Le graphique révèle des profils saisonniers différenciés selon les villes. Paris, Lyon et Lille présentent une structure similaire : l'effet saisonnier est positif le week-end (pics le samedi et dimanche). L'amplitude de cet effet varie toutefois selon la taille de la ville, Paris affichant l'effet le plus marqué, suivi de Lyon, puis de Lille. Marseille se distingue nettement des trois autres métropoles : son pic d'effet saisonnier se situe le dimanche, avec un effet moins prononcé le samedi, ce qui suggère un usage davantage tourné vers les loisirs et le tourisme balnéaire.

En conclusion la composante saisonnière n'est pas identique entre les quatre villes. Marseille

se différencie nettement de Paris, Lyon et Lille, tant par la forme que par l'amplitude de sa saisonnalité hebdomadaire.